
31 Number-Theoretic Algorithms

Number theory was once viewed as a beautiful but largely useless subject in pure mathematics. Today number-theoretic algorithms are used widely, due in large part to the invention of cryptographic schemes based on large prime numbers. These schemes are feasible because we can find large primes quickly, and they are secure because we do not know how to factor the product of large primes (or solve related problems, such as computing discrete logarithms) efficiently. This chapter presents some of the number theory and related algorithms that underlie such applications.

We start in Section 31.1 by introducing basic concepts of number theory, such as divisibility, modular equivalence, and unique prime factorization. Section 31.2 studies one of the world's oldest algorithms: Euclid's algorithm for computing the greatest common divisor of two integers, and Section 31.3 reviews concepts of modular arithmetic. Section 31.4 then explores the set of multiples of a given number a , modulo n , and shows how to find all solutions to the equation $ax = b \pmod{n}$ by using Euclid's algorithm. The Chinese remainder theorem is presented in Section 31.5. Section 31.6 considers powers of a given number a , modulo n , and presents a repeated-squaring algorithm for efficiently computing $a^b \pmod{n}$, given a , b , and n . This operation is at the heart of efficient primality testing and of much modern cryptography, such as the RSA public-key cryptosystem described in Section 31.7. We wrap up in Section 31.8, which examines a randomized

primality test. This test finds large primes efficiently, an essential step in creating keys for the RSA cryptosystem.

Size of inputs and cost of arithmetic computations

Because we'll be working with large integers, we need to adjust how to think about the size of an input and about the cost of elementary arithmetic operations.

In this chapter, a “large input” typically means an input containing “large integers” rather than an input containing “many integers” (as for sorting). Thus, the size of an input depends on the *number of bits* required to represent that input, not just the number of integers in the input. An algorithm with integer inputs $a_1, a_2, \dots, a_k$ is a **polynomial-time algorithm** if it runs in time polynomial in $\lg a_1, \lg a_2, \dots, \lg a_k$, that is, polynomial in the lengths of its binary-encoded inputs.

Most of this book considers the elementary arithmetic operations (multiplications, divisions, or computing remainders) as primitive operations that take one unit of time. Counting the number of such arithmetic operations that an algorithm performs provides a basis for making a reasonable estimate of the algorithm's actual running time on a computer. Elementary operations can be time-consuming, however, when their inputs are large. It thus becomes appropriate to measure how many **bit operations** a number-theoretic algorithm requires. In this model, multiplying two β -bit integers by the ordinary method uses $\Theta(\beta^2)$ bit operations. Similarly, dividing a β -bit integer by a shorter integer or taking the remainder of a β -bit integer when divided by a shorter integer requires $\Theta(\beta^2)$ time by simple algorithms. (See Exercise 31.1-12.) Faster methods are known. For example, a simple divide-and-conquer method for multiplying two β -bit integers has a running time of $\Theta(\beta^{\lg 3})$, and $O(\beta \lg \beta \lg \lg \beta)$ time is possible. For practical purposes, however, the $\Theta(\beta^2)$ algorithm is often best, and we use this bound as a basis for our analyses. In this chapter, we'll usually analyze algorithms in terms of both the number of arithmetic operations and the number of bit operations they require.